CUET PG

DAILY CLASS NOTES

Computer Science

Operating System

Lecture – 05

Process Synchronization and Critical Section



Process Synchronization and Critical Section

Topics to be Covered

1. Process Synchronization
2. Critical Section Definition
3. Requirement of Critical Section Solution
4. Critical Section Algorithms

1. Process Synchronization

- It is a **mechanism** to ensure **systematic and orderly sharing of resources** among multiple concurrent processes.
- It ensures that **no two processes access the same shared resource simultaneously**, preventing conflicts and data inconsistency.

2. Critical Section

- The **critical section** refers to that part of a program where the **process accesses shared resources** (like variables, files, databases, or devices).
- Since shared resources are non-shareable, **only one process at a time** should be allowed to execute its critical section.

3. Requirements of a Good Critical Section Solution

An ideal critical section solution must satisfy three essential requirements:

(a) Mutual Exclusion

- At any given time, **only one process** can execute inside its critical section.
- If one process is executing its critical section, all other processes must be prevented from entering theirs.

(b) Progress

- If **no process is currently in its critical section**, and one or more processes wish to enter, the **selection of the next process should not be delayed indefinitely**.
- The decision of which process can enter next must be made within a **finite time**, ensuring progress.
- This guarantees that the system **does not remain idle unnecessarily** when work can proceed.

(c) Bounded Waiting

- There must exist a **finite upper bound** on the number of times that other processes can enter their critical sections **after a process has made a request** to enter its own.
- This prevents **starvation**, i.e., no process waits indefinitely to enter its critical section.
- Normally, this upper bound is considered **1**, meaning every process gets a fair chance.

- **General Structure of a Critical Section Solution**

The general structure of code for handling critical sections follows a repeated cycle:

```
do {
   Entry Section
   Critical Section
   Exit Section
   Remainder Section
} while(1);
```

- **Components of Critical Section Solution**
  1. **Entry Section**
     - The code segment is executed **when a process requests to enter** its critical section.
     - It checks whether entry is allowed based on mutual exclusion conditions.
  2. **Critical Section**
     - The part where the process **accesses shared resources**.
     - Only one process can execute this section at a time.
     - Provides **exclusive (non-shareable)** access to shared data.
  3. **Exit Section**
     - Executed **immediately after** a process exits its critical section.
     - It **updates status flags** or signals that another waiting process can now enter the critical section.
  4. **Remainder Section**
     - The **remaining portion** of a process outside the entry, critical, and exit sections.
     - It represents normal execution where the process **is not competing** for the critical section.
     - Processes in this section **are not considered candidates** for entry into the critical section.

4. Critical Section Algorithms

- The goal of all critical section algorithms is to ensure that **the three conditions (Mutual Exclusion, Progress, Bounded Waiting)** are met.

Therefore, any correct algorithm must guarantee:

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

Algorithm 1: "I finished with it. Now you're it."

Description

- Works for two processes only – say P_0 and P_1.
- Uses a **shared integer variable "turn"** that indicates whose turn it is to enter the critical section.
  int turn = 0; // 0 or 1, indicating process allowed to enter

Working

- If turn = 0, then process **P_0** can enter; otherwise **P_2** enters.
- After finishing, the process sets the turn value to the other process.

Conditions Met

- **Mutual Exclusion**
- **Bounded Waiting**
- **Progress not satisfied** — because the algorithm depends on the value of "turn," so if one process stops working, the other may wait indefinitely.

Limitation:

- Does **not meet the Progress requirement**, so not a complete solution.

Algorithm 2:

"Let me have it after you finish with it."

Description

- Also designed for **two processes** (P_0 and P_1).
- Uses a **Boolean array "flag"** to indicate if a process is ready to enter its critical section.
  typedef enum {false, true} Boolean;
  Boolean flag[2];
- Each process sets its flag to true when it wants to enter and checks the other's flag before entering.

Working

- If both processes try to enter simultaneously, one waits until the other finishes.
- Ensures mutual exclusion using the "flag" variable.

Conditions Met

- **Mutual Exclusion**
- **Bounded Waiting**
- **Progress not satisfied**

Limitation:

- Progress condition fails, meaning a process might unnecessarily wait even when the other is outside its critical section.

Algorithm 3: Peterson's Algorithm

Description

- This is the **most efficient and correct software-based solution** for critical section problems.
- Works for **two or more cooperating processes**.
- It uses **two shared variables**:
  - turn → indicates whose turn it is to enter.
  - flag[i] → indicates if process Pi wants to enter its critical section.

Properties

Meets **all three requirements** of an ideal critical section solution:

1. Mutual Exclusion
2. Progress
3. Bounded Waiting

Hence, it is considered a generic synchronization algorithm used for coordinating concurrent processes.

Summary of Process Synchronization & Critical Section)

Concept	Description / Key Points
Process Synchronization	Mechanism ensuring safe access to shared resources among concurrent processes.
Critical Section	Code section accessing shared data; must be executed by only one process at a time.
Requirements	Mutual Exclusion, Progress, and Bounded Waiting.
Structure	Entry Section → Critical Section → Exit Section → Remainder Section (repeated in loop).
Algorithm 1	Uses variable "turn"; satisfies Mutual Exclusion & Bounded Waiting only.
Algorithm 2	Uses a Boolean "flag"; also misses the Progress condition.
Peterson's Algorithm	Uses both "turn" & "flag"; satisfies all three — Mutual Exclusion, Progress, and Bounded Waiting.

प्रोसेस सिंक्रोनाइज़ेशन और क्रिटिकल सेक्शन

कवर किए जाने वाले विषय

1. प्रोसेस सिंक्रोनाइज़ेशन
2. क्रिटिकल सेक्शन की परिभाषा
3. क्रिटिकल सेक्शन समाधान की आवश्यकता
4. क्रिटिकल सेक्शन एल्गोरिदम

1. प्रोसेस सिंक्रोनाइज़ेशन

- यह कई समवर्ती प्रक्रियाओं के बीच संसाधनों के व्यवस्थित और क्रमबद्ध साझाकरण को सुनिश्चित करने का एक तंत्र है।
- यह सुनिश्चित करता है कि कोई भी दो प्रक्रियाएँ एक साथ एक ही साझा संसाधन तक पहुँच प्राप्त न करें, जिससे संघर्ष और डेटा विसंगति को रोका जा सके।

2. क्रिटिकल सेक्शन

- क्रिटिकल सेक्शन एक प्रोग्राम के उस भाग को संदर्भित करता है जहाँ प्रोसेस साझा संसाधनों (जैसे कि चर, फाइलें, डेटाबेस या उपकरण) तक पहुँच प्राप्त करती है।
- चूँकि साझा संसाधन गैर-साझाकरण योग्य होते हैं, इसलिए एक समय में केवल एक ही प्रोसेस को अपने क्रिटिकल सेक्शन को निष्पादित करने की अनुमति दी जानी चाहिए।

3. एक अच्छे क्रिटिकल सेक्शन समाधान की आवश्यकताएँ

(a) पारस्परिक अपवर्जन

- किसी भी समय, केवल एक ही प्रोसेस अपने क्रिटिकल सेक्शन के भीतर निष्पादन कर सकती है।
- यदि एक प्रोसेस अपने क्रिटिकल सेक्शन का निष्पादन कर रही है, तो अन्य सभी प्रक्रियाओं को उनके क्रिटिकल सेक्शन में एंट्रीकरने से रोका जाना चाहिए।

(b) प्रोग्रेस

- यदि वर्तमान में कोई भी प्रोसेस अपने क्रिटिकल सेक्शन में नहीं है, और एक या अधिक प्रक्रियाएँ एंट्रीकरना चाहती हैं, तो अगली प्रोसेस के चयन में अनिश्चित काल के लिए देरी नहीं होनी चाहिए।
- अगली प्रोसेस कौन सी एंट्रीकर सकती है, इसका निर्णय एक सीमित समय के भीतर लिया जाना चाहिए, जिससे कार्य की प्रोग्रेस सुनिश्चित हो सके।
- यह गारंटी देता है कि जब कार्य आगे बढ़ सकता है, तब सिस्टम अनावश्यक रूप से खाली (idle) नहीं रहता है।

(c) बॉउनडेड वैटिंग

- किसी प्रोसेस द्वारा अपने क्रिटिकल सेक्शन में एंट्रीकरने का अनुरोध करने के बाद, अन्य प्रक्रियाओं के अपने क्रिटिकल सेक्शनों में एंट्रीकरने की संख्या पर एक सीमित ऊपरी सीमा होनी चाहिए।
- यह स्टारवेशन को रोकता है, अर्थात कोई भी प्रोसेस अपने क्रिटिकल सेक्शन में एंट्रीकरने के लिए अनिश्चित काल तक प्रतीक्षा नहीं करती है।
- सामान्यतः इस ऊपरी सीमा को 1 माना जाता है, जिसका अर्थ है कि प्रत्येक प्रोसेस को एक निष्पक्ष अवसर मिलता है।

- **क्रिटिकल सेक्शन समाधान की सामान्य संरचना**

क्रिटिकल सेक्शनों को संभालने के लिए कोड की सामान्य संरचना एक दोहराए गए चक्र का पालन करती है:

```
do {
   Entry Section
   Critical Section
   Exit Section
   Remainder Section
} while(1);
```

- **क्रिटिकल सेक्शन समाधान के घटक**

1. **एंट्री सेक्शन**
   - जब कोई प्रोसेस अपने क्रिटिकल सेक्शन में एंट्रीकरने का अनुरोध करती है, तब कोड का यह भाग निष्पादित होता है।
   - यह जाँच करता है कि पारस्परिक अपवर्जन की शर्तों के आधार पर एंट्रीकी अनुमति है या नहीं।
2. **क्रिटिकल सेक्शन**
   - यह वह भाग है जहाँ प्रोसेस साझा संसाधनों तक पहुँच (एक्सेस) प्राप्त करती है।
   - एक समय में केवल एक ही प्रोसेस इस खंड को निष्पादित कर सकती है।
   - यह साझा डेटा तक अनन्य (गैर-साझाकरण योग्य) पहुँच प्रदान करता है।
3. **एग्जिट सेक्शन**
   - किसी प्रोसेस द्वारा अपने क्रिटिकल सेक्शन से बाहर निकलने के तुरंत बाद इसे निष्पादित किया जाता है।
   - यह स्थिति फ्लैग्स को अद्यतन करता है या संकेत देता है कि अब दूसरी प्रतीक्षा कर रही प्रोसेस क्रिटिकल सेक्शन में एंट्रीकर सकती है।
4. **रिमेंडर सेक्शन**
   - प्रवेश, महत्वपूर्ण और एग्जिट सेक्शन के बाहर की प्रोसेस का शेष हिस्सा।
   - यह सामान्य निष्पादन का प्रतिनिधित्व करता है जहाँ प्रोसेस क्रिटिकल सेक्शन के लिए प्रतिस्पर्धा नहीं कर रही होती है।
   - इस खंड की प्रक्रियाओं को क्रिटिकल सेक्शन में एंट्रीके लिए उम्मीदवार नहीं माना जाता है।

4. क्रिटिकल सेक्शन एल्गोरिदम

- सभी क्रिटिकल सेक्शन एल्गोरिदम का लक्ष्य यह सुनिश्चित करना है कि तीन शर्तें (पारस्परिक अपवर्जन, प्रोग्रेस, बॉउनडेड वैटिंग) पूरी हों।

अतः किसी भी सही एल्गोरिदम को निम्नलिखित की गारंटी देनी चाहिए:

1. पारस्परिक अपवर्जन
2. प्रोग्रेस
3. बॉउनडेड वैटिंग

एल्गोरिदम 1: "मेरा कार्य समाप्त हुआ। अब आपकी बारी है।"

विवरण

- यह केवल दो प्रक्रियाओं के लिए कार्य करता है – मान लीजिए P_0 और P_1
- इसमें एक साझा पूर्णांक चर "टर्न"का उपयोग किया जाता है जो यह संकेत देता है कि क्रिटिकल सेक्शन में एंट्रीकरने की अब किसकी बारी है।

```
int turn = 0; // 0 or 1, indicating process allowed to enter
```

कार्यपद्धति

- यदि turn == 0 है, तो तो प्रोसेस $\mathbf{P_0}$ एंट्रीकर सकती है; अन्यथा $\mathbf{P_1}$ एंट्रीकरती है।
- अपना कार्य समाप्त करने के बाद, प्रोसेस बारी के मान को दूसरी प्रोसेस के लिए निर्धारित कर देती है।

पूरी होने वाली शर्तें

- **पारस्परिक अपवर्जन**
- **बॉउनडेड वैटिंग**
- **प्रोग्रेस संतुष्ट नहीं है** — क्योंकि यह एल्गोरिदम पूरी तरह से "टर्न" के मान पर निर्भर करता है, इसलिए यदि एक प्रोसेस काम करना बंद कर देती है, तो दूसरी प्रोसेस अनिश्चित काल तक प्रतीक्षा कर सकती है।

सीमा:

- यह प्रोग्रेस की आवश्यकता को पूरा नहीं करता है, इसलिए यह एक पूर्ण समाधान नहीं है।

एल्गोरिदम 2:

"जब आपका कार्य समाप्त हो जाए, तब मुझे इसे लेने दें।"

विवरण

- यह भी दो प्रक्रियाओं (P_0 और P_1) के लिए बनाया गया है।
- यह एक बूलियन सरणी 'फ्लैग' का उपयोग करता है जो यह संकेत देता है कि कोई प्रोसेस अपने क्रिटिकल सेक्शन में एंट्रीकरने के लिए तैयार है या नहीं।

  typedef enum {false, true} Boolean;
  Boolean flag[2];
- प्रत्येक प्रोसेस जब एंट्रीकरना चाहती है, तब वह अपने फ्लैग को ट्रू पर निर्धारित करती है और एंट्रीकरने से पहले दूसरी प्रोसेस के फ्लैग की जाँच करती है।

कार्यपद्धति

- यदि दोनों प्रक्रियाएँ एक साथ एंट्रीकरने का प्रयास करती हैं, तो एक प्रोसेस तब तक प्रतीक्षा करती है जब तक कि दूसरी प्रोसेस अपना कार्य समाप्त न कर ले।
- यह "फ्लैग" चर (वेरिएबल) का उपयोग करके पारस्परिक अपवर्जन सुनिश्चित करता है।

पूरी होने वाली शर्तें

- **पारस्परिक अपवर्जन**
- **बॉउनडेड वैटिंग**
- **प्रोग्रेस संतुष्ट नहीं है**

सीमा:

- प्रोग्रेस की शर्त विफल हो जाती है, जिसका अर्थ है कि एक प्रोसेस को अनावश्यक रूप से प्रतीक्षा करनी पड़ सकती है, भले ही दूसरी प्रोसेस अपने क्रिटिकल सेक्शन से बाहर हो।

एल्गोरिदम 3: पीटरसन का एल्गोरिदम

विवरण

- यह क्रिटिकल सेक्शन समस्याओं के लिए सबसे कुशल और सही सॉफ्टवेयर-आधारित समाधान है।
- यह दो या दो से अधिक सहकारी प्रक्रियाओं के लिए कार्य करता है।
- यह दो साझा चरों का उपयोग करता है:
  - turn → यह संकेत देता है कि एंट्रीकरने की अब किसकी बारी है।.
  - flag[i] → यह संकेत देता है कि क्या प्रोसेस Pi अपने क्रिटिकल सेक्शन में एंट्रीकरना चाहती है।

गुणधर्म

यह एक आदर्श क्रिटिकल सेक्शन समाधान की **तीनों आवश्यकताओं** को पूरा करता है:

1. पारस्परिक अपवर्जन
2. प्रोग्रेस
3. बॉउनडेड वैटिंग

इसलिए, इसे समवर्ती प्रक्रियाओं के बीच समन्वय स्थापित करने के लिए उपयोग किया जाने वाला एक सामान्य सिंक्रोनाइज़ेशन एल्गोरिदम माना जाता है।

प्रोसेस सिंक्रोनाइज़ेशन और क्रिटिकल सेक्शन का सारांश

अवधारणा	**विवरण / मुख्य बिंदु**
प्रोसेस सिंक्रोनाइज़ेशन	समवर्ती प्रक्रियाओं के बीच साझा संसाधनों तक सुरक्षित पहुँच सुनिश्चित करने वाला तंत्र।
क्रिटिकल सेक्शन	साझा डेटा तक पहुँचने वाला कोड खंड; इसे एक समय में केवल एक ही प्रोसेस द्वारा निष्पादित किया जाना चाहिए।
आवश्यकताएँ	पारस्परिक अपवर्जन, प्रोग्रेस, और बॉउनडेड वैटिंग ।
संरचना	एंट्री सेक्शन — क्रिटिकल सेक्शन — एग्जिट सेक्शन —रिमेंडर सेक्शन (लूप में दोहराया गया)।
एल्गोरिदम 1	"टर्न" (turn) चर का उपयोग करता है; केवल पारस्परिक अपवर्जन और बॉउनडेड वैटिंग को संतुष्ट करता है।
एल्गोरिदम 2	एक बूलियन फ्लैग का उपयोग करता है; यह भी प्रोग्रेस की शर्त को पूरा नहीं कर पाता है।
पीटरसन का एल्गोरिदम	"टर्न" और "फ्लैग" दोनों का उपयोग करता है; तीनों शर्तों — पारस्परिक अपवर्जन, प्रोग्रेस, और बॉउनडेड वैटिंग को संतुष्ट करता है।



PW Web/App: https://smart.link/7wwosivoicgd4

Library- https://smart.link/sdfez8ejd80if